DATABASE SYSTEMS

Complete Study Notes | Lectures 1 to 15

Easy explanations, clear examples, memory cues, comparison tables, SQL patterns, and clean diagrams.

How to use these notes
Read each concept in order. Study the diagrams by tracing the arrows and relationships. Then use the quick-recall section and self-test questions to check whether you can explain the idea without looking.

Prepared from the 15 uploaded Database Systems lecture slide decks.

Study roadmap

The notes follow the sequence of your lecture slides. Repeated introductory material is combined where it helps learning, while the key terms and topics from each lecture are retained.

Lecture	Topics
01	Database, DBMS, data and information
02	File processing systems vs DBMS
03	Database approach, DFDs and database environment
04	Bus garage DFD, schema and three-level architecture
05	Data independence, DDL/DML and data models
06	Hierarchical and network database models
07	Object-oriented and relational database models
08	DBMS functions, architecture and metadata
09	Relational model, domains, keys and surrogate keys
10	Integrity rules and SQL constraints
11	Views and indexes
12	Conceptual schema design and relationships
13	Attribute types, weak entities and entity vs attribute
14	Building an ERD from a written description
15	Entity classification and ERD refinement

Important distinction
A database is the organized collection of data. A DBMS is the software that defines, creates, maintains, protects, and provides access to that database.

Lecture 1 | Introduction to databases

A database is a shared collection of logically related data, together with a description of that data, designed to meet an organization's information needs.

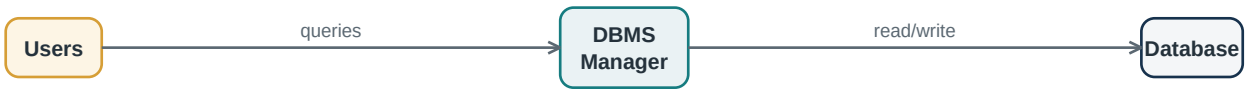
Understand it with an example

A university may store students, courses, lecturers, enrollments, fees, and results. These facts are related, so they are kept in an organized shared database instead of being scattered across unrelated files.

- **Shared:** different authorized users and applications can use the same data.
- **Logically related:** the data belongs together because it describes connected parts of an organization.
- **Description of data:** the database also keeps definitions such as table names, column names, data types, and rules. This is metadata.
- **Information needs:** the database is designed to answer the organization's questions and support its work.

Database vs DBMS

Database	DBMS
The organized collection of related data.	The software used to manage the database.
Example: student records and course records.	Examples: Microsoft SQL Server, MySQL, Oracle.
Contains the stored facts and their structure.	Defines, creates, maintains, and controls access to the data.



The DBMS controls and manages access to stored data.

Data, information and metadata

Data means raw facts, such as 82, 76, and 91. **Information** is data processed into a meaningful result, such as the student's average mark being 83. **Metadata** is data about data, such as the fact that Marks is an integer column with an allowed range.

Main components of a DBMS

- **DDL (Data Definition Language):** describes the database structure, data types, and constraints. Think: define the design.
- **DML (Data Manipulation Language):** reads and changes stored data. Think: work with the data.
- **Controlled access:** security, integrity, concurrency control, and recovery.

Memory cue
DDL = Define the database. DML = Manipulate the data. DBMS = Manage the whole database.

Lecture 2 | File processing systems vs DBMS

A traditional file-based system is a collection of application programs. Each program performs a task and manages its own data files. For example, payroll has its own file, customer accounts have another, and customer mailings may keep a third copy of customer details.

Why file-based systems cause problems

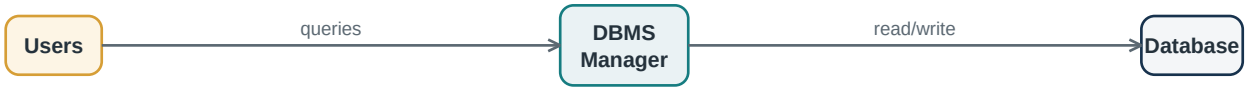
Problem	Easy explanation	Example
Data isolation	Files are separated and difficult to combine.	Sales and accounts store information separately.
Data redundancy	The same fact is stored more than once.	A customer's address appears in three files.
Inconsistency	Copies may disagree after an update.	One file has the new address, another has the old one.
Data dependence	Programs are tied to a particular file structure.	Changing a field may require changing the program.
Incompatible formats	Different programs may store data differently.	One file uses a different layout from another.
Fixed queries	Programs provide only the reports they were designed for.	A new question may require a new program.

Data redundancy and inconsistency

Suppose a customer changes their address. If the address is stored in the order file, account file, and mailing-list file, all copies must be updated. If one is missed, the organization now has conflicting versions of the same fact.

How the database approach improves this

A DBMS provides a central repository. Different applications use the shared database through the DBMS instead of each maintaining a separate private copy.



The DBMS controls and manages access to stored data.

Components of a database environment

Component	What it means
Hardware	Computers, storage devices, and network equipment.
Software	DBMS, application programs, and query software.
Data	The organized facts, schema, and subschemas.
Procedures	Rules for designing, accessing, and using the database.
People	Database administrators, designers, developers, and users.

Advantages of the database approach

- Controls redundancy and improves consistency.
- Allows data sharing across the organization.

- Improves data integrity, validity, security, and accessibility.
- Enforces standards and supports centralized management.
- Improves productivity, concurrency, backup, and recovery.
- Data independence makes maintenance easier.

Disadvantages

- More complex and larger than simple file systems.
- DBMS software and extra hardware cost money.
- Converting old systems to a DBMS can be expensive.
- A DBMS may add processing overhead or be slow in some production situations.
- A central failure can affect many users, so its impact can be high.

Exam comparison
File system: each application manages its own files. DBMS: applications share centrally managed data through DBMS software.

Lecture 3 | Organizing data and Data Flow Diagrams

Entity, attribute and relationship

Term	Meaning	Example
Entity	A distinguishable real-world object, person, place, thing, concept, or event.	Customer
Attribute	A property that describes an entity.	CustomerName, Address
Relationship	An association between entities.	Customer places Order

Quick test
Ask: What thing are we storing information about? That is likely an entity. What fact describes it? That is an attribute. How are two things connected? That is a relationship.

What is a DFD?

A Data Flow Diagram (DFD) shows how data moves through a system. It focuses on data entering, being processed, being stored, and leaving the system. It is not a flowchart of program instructions.

DFD element	What it represents	Naming rule
External entity	A person or outside system that sends or receives data.	Noun
Process	An activity that transforms or handles data.	Verb phrase
Data flow	Data moving between elements.	Name the data
Data store	A place where data is kept.	Noun



External entity -> Process -> Data store

Levels of a DFD

Level	Purpose	What to show
Context diagram	Shows the whole system as one process and its external entities.	System boundary and outside parties.
Level 0	Breaks the whole system into major manageable processes.	Main processes and high-level flows.
Level 1 to n	Breaks a Level 0 process into more detail.	Sub-processes, actual flows, and data stores.

How to create a DFD

1. List the activities. 2. Draw the context-level diagram. 3. Draw Level 0 and identify major sub-processes. 4. Draw Level 1 or further levels to show details and data stores. 5. Check the diagram against DFD rules.

Lemonade stand example

Activities include taking a customer order, serving the product, collecting payment, producing and storing lemonade, ordering raw materials, paying vendors, and paying employees. These can be grouped into Sale, Production, Procurement, and Payroll.

DFD level	Lemonade stand example
Context	One process called Lemonade System exchanges order/payment/product information with Customer, Employee, and Vendor.
Level 0	Break the system into Sale, Production, Procurement, and Payroll.
Level 1	Break Sale into Record Order, Receive Payment, and Produce Sales Forecast. Break other major processes into their own sub-processes and connect data stores.

DFD memory cue
Context = one big process. Level 0 = main processes. Level 1 = detailed sub-processes and data stores.

Lecture 4 | Bus garage DFD, schema and subschemas

Bus garage repair example

Buses arrive at a garage for repair. A mechanic and helper perform the repair, record the reason, parts used, labor, and total cost on a Shop Repair Order. The information is then used by Accounting for billing, Inventory Management for parts monitoring, and the Supervisor for performance review.

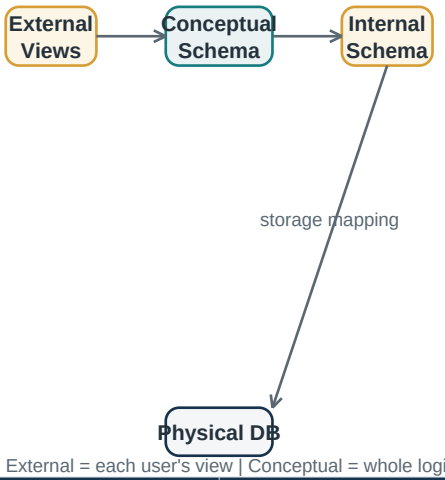
DFD category	Bus garage example
External entities	Bus, mechanic, helper, supervisor, inventory management system, accounting department.
Main process	Perform repairs and store repair information.
Sub-processes	Record bus ID and repair reason; determine required parts; perform repair; calculate parts and total cost; record labor hours and cost.
Data stores	Personnel file, repairs file, bus master list, parts list.
Data flows	Repair order, bus record, parts record, employee timecard, invoices.

Schema and subschema

Schema: the complete logical description or catalog of all data in the database. It describes the overall structure.

Subschema: a portion of the database structure relevant to a particular user or application. It supports a customized view.

Three-level database architecture



External = each user's view | Conceptual = whole logical database | Internal = storage details

Level	Meaning	Example
External	A particular user's view of the database.	A cashier sees customer name and order total, not payroll data.
Conceptual	The overall logical structure and relationships of the entire database.	The full set of entities, attributes, and relationships.
Internal	How data is physically organized and stored.	Files, storage structures, and physical organization.

Why use three levels?

- Different users can have different views of the same shared data.
- Users do not need to understand physical storage details.

- The DBA can change storage structures without forcing every user view to change.
- The overall logical structure can be managed separately from physical storage.

Remember the order
External = user view. Conceptual = complete logical design. Internal = physical storage.

Lecture 5 | Data independence, languages and models

Data independence

Data independence means that a change at one database level should not force unnecessary changes at the level above it.

Type	Definition in simple words	Example
Physical data independence	The conceptual schema is protected from changes to the internal schema.	Changing file organization or adding an index should not change the logical tables users work with.
Logical data independence	External schemas are protected from changes to the conceptual schema.	Adding a new attribute to the overall database should not break every user's existing view.

Most important distinction
Physical independence: Internal changes should not affect the conceptual level. Logical independence: Conceptual changes should not affect external views.

DDL vs DML

DDL	DML
Defines or modifies the database structure, schema, and subschemas.	Reads and changes the data stored in the database.
Think: design the tables and rules.	Think: insert, read, update, delete rows.
Examples: CREATE, ALTER, DROP.	Examples: SELECT, INSERT, UPDATE, DELETE.

What is a data model?

A data model is a collection of concepts used to describe data, relationships between data, and constraints on the data.

- **Structural part:** rules for constructing the database structure.
- **Manipulative part:** operations allowed on the data.
- **Integrity rules:** rules that help keep data accurate.

Conceptual vs implementation models

Conceptual model	Implementation model
Describes the logical nature of the data and what entities exist. It is a design blueprint.	Describes how data is represented in a database system.
Example: Entity-Relationship model.	Examples: Hierarchical, Network, Relational, Object-oriented.

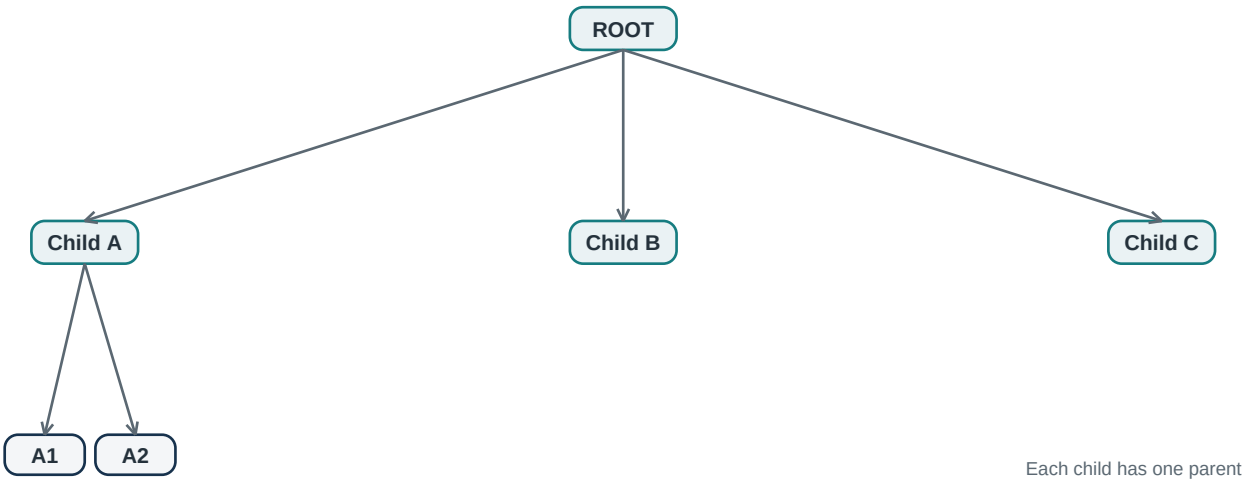
Evolution of database systems

The slides outline this broad progression: flat files (1960s to 1980s), hierarchical and network systems (1970s to 1990s), relational systems (1980s to present), object-oriented and object-relational systems (1990s to present), and data warehousing and web-enabled systems (1980s/1990s to present). Later slides also show NoSQL, JSON, and XML.

Lecture 6 | Hierarchical and network models

Hierarchical database model

The hierarchical model organizes data like an upside-down tree. One root branches into parent and child records.



Each child has one parent

- One parent can have many children.
- Each child has only one parent.
- Links between parent and child records use pointers.
- It naturally represents one-to-many (1:M) relationships.

Example: One bank customer account can have many transactions. The account is the parent and the transactions are children.

Advantages

- Conceptually simple parent-child structure.
- Security can be enforced uniformly.
- Supports data integrity and data independence.
- Efficient for fixed 1:M relationships and large numbers of repeated transactions.

Disadvantages

- Complex implementation and database design.
- Difficult to manage and lacks standards in the slide discussion.
- Lacks structural independence.
- Pointer-based application programming can be complex.
- Limited mainly to 1:M relationships.

Network database model

The network model extends the hierarchical idea. A record can have multiple parents, so the structure can represent more complex connections.

- It uses owner records and member records. A set describes the relationship between them.
- A member can have several owners.
- The slides identify network schema, subschema, and database management language as important components.
- The hierarchical model is a subset of the network model.

Advantages and disadvantages

Advantages	Disadvantages
Conceptually similar to hierarchical model; supports more relationship types; flexible data access; promotes integrity and data independence; conforms to standards.	System complexity; described in the slides as designed more for programmers than end users; lacks structural independence.

Compare quickly

Hierarchical: each child has one parent. Network: a member can have multiple owners/parents.

Lecture 7 | Object-oriented and relational models

Object-oriented database model (OODB)

An object-oriented database stores data as objects. Objects are instances of classes. A class groups the data fields and operations that belong to that kind of object.

For example, a Clerk class may contain name, address, ID, and salary fields, along with operations such as get_name(), get_address(), and annual_salary().

Object data may also contain nested objects and arrays. A book object, for example, can contain a title, a list of authors, a publication date, page count, and a nested publisher object.

Advantages and disadvantages

- Advantages listed in the slides: real-world modeling, model simplicity, and reduced data size.
- Disadvantages listed in the slides: lack of SQL query language and lack of automatic constraints. The slide text appears to contain a wording typo for the latter.

Relational database model (RDB)

A relational database is presented to users as a collection of tables. The RDBMS handles the physical details behind the scenes. Tables contain rows and columns, and tables are connected through common data such as keys.

Relational term	Meaning
Relation	A table.
Tuple	A row in a table.
Attribute	A column in a table.
Domain	The allowed values for an attribute.

The relational model supports 1:1, 1:M, and M:N relationships.

Advantages

- Structural independence: users do not need to know the data access path.
- Conceptual simplicity and easier database design and management.
- SQL supports ad hoc queries, meaning users can ask new questions without writing a separate application for every query.
- Powerful database management capabilities.

Disadvantages

- May require substantial hardware and system software resources.
- Poor design and implementation are still possible.
- May create isolated pockets of information if databases are not integrated properly.

Model families from the slides

Family	Models mentioned
Object-based	Entity-relationship, semantic, functional, object-oriented.
Record-based	Relational, network, hierarchical.
Physical	Unifying and frame-memory models.

Memory cue
Hierarchical = tree. Network = multiple parents. Object-oriented = objects and classes. Relational = tables.

Lecture 8 | DBMS functions, architecture and metadata

Functions of a DBMS

- Stores, retrieves, and updates data.
- Provides a user-accessible catalog.
- Supports transactions.
- Controls concurrent access by multiple users.
- Provides recovery after failures.
- Controls authorization and security.
- Supports data communications.
- Enforces integrity rules.
- Promotes data independence and provides utility services.

DBMS architecture: understand the jobs

Component	Job in simple wording
DDL processor	Processes commands that define or change database structure.
DML processor	Processes data manipulation commands.
Query processor / command processor	Receives and processes user commands or queries.
Query optimizer	Chooses an efficient strategy for executing a query.
Authorization control	Checks whether the user is allowed to perform the action.
Integrity checker	Checks whether integrity constraints are satisfied.
Transaction manager	Coordinates transaction processing.
Scheduler	Manages concurrent operations so they do not interfere incorrectly.
Buffer manager	Moves data between primary memory and secondary storage.
Recovery manager	Helps restore the database after a failure.
Dictionary manager	Manages the data dictionary or catalog.
File manager / access methods	Support access to stored database files and records.

A simple query journey

A user sends a query. The DBMS checks permissions and integrity rules, processes and optimizes the command, coordinates the transaction and concurrent work, accesses data through storage management, and returns the result. The slides also show a flow involving Query, Transaction, Journal, Buffered data, and the operating system.

Metadata

Metadata means data about data. It helps users and the DBMS understand what the actual stored data represents and how it is organized.

Type	What it describes
Technical metadata	Database and table names, sizes, data types, attributes, primary keys, foreign keys, indexes, and constraints.

Type	What it describes
Business metadata	Business ownership, policies, rules, regulations, and meaning of data.
Operational metadata	Operational details about data handling and use. The slide description overlaps with business metadata.
Descriptive metadata	Details describing a file or item, such as title, author, date, size, or publication information.

System catalog / data dictionary: a repository of metadata. It can store authorized user names, database item names, data types, sizes, constraints, and authorization information.

Remember
Metadata describes the database. The system catalog stores that description.

Lecture 9 | Relational model, keys and constraints

How to read a relation

A relation is a table. Its rows are tuples and its columns are attributes. The slides use two measurements: **cardinality** is the number of rows, while **degree** is the number of columns.

Term	Meaning	Example
Relation	A table.	STUDENT
Tuple	One row / record.	One student's record.
Attribute	One column / property.	StudentID or Name.
Domain	The set of allowed values for an attribute.	Age may be restricted to positive integers.

Properties of relations

- Every relation has a distinct name.
- Each cell contains exactly one atomic value. No repeating groups inside a cell.
- Attribute names are distinct, and values in a column come from the same domain.
- The order of columns does not matter.
- Rows are distinct. Duplicate tuples are not allowed.
- The order of rows does not matter.

Keys: the identification system

Key type	Definition	Example / memory cue
Super key	One or more attributes that uniquely identify a row.	StudentID, or StudentID + Name.
Candidate key	A minimal super key. No attribute can be removed while keeping uniqueness.	StudentID if it alone uniquely identifies a student.
Primary key	The candidate key selected as the main identifier.	StudentID.
Composite key	A key made from more than one attribute.	(StudentID, CourseID) in Enrollment.
Surrogate key	An artificial key generated by the system, often an auto-incrementing number.	EnrollmentID = 1001.

Key hierarchy
Every candidate key is a super key, but not every super key is a candidate key. A primary key is one chosen candidate key.

Choosing a primary key

- It should identify an entity efficiently.
- Its value should remain stable throughout the entity's life.
- Avoid choosing a value that changes frequently.

Surrogate keys

A surrogate key is an artificial identifier used as the primary key instead of a real-world attribute. It is often automatically generated and incremented.

Advantages	Disadvantages
Avoids complicated composite primary keys; can make retrieval and management more flexible.	Has no business meaning; does not by itself stop users from inserting duplicate real-world records.

SQL examples

Single-column primary key:

```
CREATE TABLE Persons (
PersonID int IDENTITY(1,1) PRIMARY KEY,
LastName varchar(255) NOT NULL,
FirstName varchar(255)
);
```

Composite primary key:

```
CREATE TABLE Enrollment (
StudentID int NOT NULL,
CourseID int NOT NULL,
PRIMARY KEY (StudentID, CourseID)
);
```

The slides show SQL Server IDENTITY(1,1) and MySQL AUTO_INCREMENT as ways to generate surrogate key values. Exact syntax depends on the DBMS.

Lecture 10 | Relational integrity and SQL constraints

A constraint is a rule that restricts the values allowed in a table. If an insert or update violates a constraint, the DBMS rejects that action.

Three integrity ideas

Integrity rule	Meaning
Entity integrity	A primary key cannot be NULL. Every row must be identifiable.
Referential integrity	A foreign key must match a candidate key in the referenced table, or be wholly NULL when NULL is allowed.
Enterprise constraints	Additional rules defined by the organization or business.

SQL constraints

Constraint	Purpose	Example
NOT NULL	A column must have a value.	StudentName cannot be NULL.
UNIQUE	Values must be distinct.	Email must not be repeated.
PRIMARY KEY	Unique identifier, combining uniqueness and non-NULL.	StudentID.
FOREIGN KEY	Connects a row to a key in another table.	Orders.PersonID references Person.PersonID.
CHECK	Value must satisfy a condition.	Age >= 18.
DEFAULT	Supplies a value when none is provided.	City defaults to 'Lahore'.
INDEX	Speeds up finding rows; it is not an integrity rule in the same sense as the others.	Index on LastName.

Foreign key example

Person is the parent table. Orders is the child table. Each order stores the PersonID of the person who placed it.

```
CREATE TABLE Person (
PersonID int PRIMARY KEY,
FirstName nvarchar(40) NOT NULL,
LastName nvarchar(40) NOT NULL
);

CREATE TABLE Orders (
OrderID int PRIMARY KEY,
OrderNumber int NOT NULL,
PersonID int,
FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
);
```

The value in Orders.PersonID must refer to an existing Person.PersonID, unless it is NULL and the design permits NULL.

Column-level vs table-level constraints

A column-level constraint is written with the column definition and applies to that column. A table-level constraint is written separately and can involve one or more columns, such as a composite primary key.

CHECK and DEFAULT examples

Age must be at least 18:

```
Age int CHECK (Age >= 18)
```

If City is omitted, use a default:

```
City varchar(255) DEFAULT 'Lahore'
```

Foreign key catch
A foreign key does not usually have to be unique. Many orders can refer to the same person. The referenced column must be a candidate key, such as a primary key or another unique key.

Lecture 11 | SQL views and indexes

Views

A view is a virtual relation. It is derived dynamically from one or more existing base tables using relational operations such as select, project, and join. A normal view stores its definition, not a separate copy of the result data.

Base relation: a named relation whose tuples are physically stored. **View:** a result derived from one or more base relations.

Why use views?

- Hide selected columns or rows from users for security.
- Give different users customized ways to see the same data.
- Simplify complex queries and joins.
- Support the external level of the three-level architecture.
- Help provide logical data independence.

SQL view example

```
CREATE VIEW [Brazil Customers] AS
SELECT CustomerName, ContactName
FROM Customers
WHERE Country = 'Brazil';

SELECT * FROM [Brazil Customers];
```

Types of views

Type	Meaning
Simple view	Usually based on one base table and straightforward columns.
Complex view	May involve multiple tables, joins, expressions, or grouping.
Inline view	A SELECT subquery used in the FROM clause of another SELECT. It behaves like a temporary result for that query.
Materialized view	Stores the retrieved result physically so it can be reused without recomputing the view each time. Also called a snapshot in the slides.

A materialized view can improve query performance, but the stored result must be synchronized with changes in the underlying tables.

Indexes

An index is a schema object that helps the DBMS find rows faster, often by using pointers and a quicker access path. It can reduce disk input/output for retrieval.

Trade-off
Indexes can speed up SELECT queries and WHERE searches, but they add work when rows are inserted or updated. Faster reading can mean slower writing.

Clustered vs non-clustered index

Clustered index	Non-clustered index
Defines the physical order of data in the table.	Stored separately from the table data and points to the rows.

Clustered index	Non-clustered index
A table's rows can have only one physical order, so there can be only one clustered index.	A table can have multiple non-clustered indexes.
The slides note that SQL Server may create a clustered index for a primary key by default, depending on the table design and existing indexes.	Like a book index: the index is separate from the content.

Creating an index

```
CREATE INDEX IndexCustomerName
ON Customer_2 (LastName ASC, FirstName ASC);
```

Indexes can be created or dropped without changing the actual table data. They change how the DBMS locates rows, not the facts stored in the rows.

Lecture 12 | Conceptual schema design and relationships

Database design steps

Step	What happens
1. Requirements analysis	Find out what users need the database to do.
2. Conceptual design	Create a high-level model, often using an ER model.
3. Schema refinement	Check consistency and improve the design, including normalization.
4. Logical design	Translate the ER model into the DBMS data model.
5. User interface design	Create external schemas or user views.
6. Physical design	Decide indexes and physical disk organization.
7. Security design	Decide who can access what and how.

ER model basics

Entity: a distinguishable real-world object described by attributes. **Entity set:** a collection of similar entities, such as all employees. Entities in the same entity set share the same type of attributes.

Relationship: an association among two or more entities. The connected entities are called participants. Example: an employee works in a department.

Degree of a relationship

Degree	Meaning	Example
Unary	One entity set participates, relating to itself.	Employee supervises Employee.
Binary	Two entity sets participate.	Student enrolls in Course.
Ternary	Three entity sets participate together.	Employee works for a Department at a Location.

Cardinality / mapping ratios

Relationship	Meaning	Example
1:1	One A is related to at most one B, and vice versa.	One supplier has one address, in the slide example.
1:M	One A can relate to many B records.	One department offers many courses.
M:1	Many A records can relate to one B.	Many products are supplied by one supplier.
M:N	Many A records can relate to many B records.	Students take many courses and each course has many students.



One department offers many courses.
Each course belongs to one department (as described in the example).

Optional and mandatory participation

- **Optional / partial participation:** an entity may exist without participating in the relationship. Example: a course may have no students enrolled yet.
- **Mandatory / total participation:** every entity in the set must participate. The slides show total participation with a double line.

Ternary relationship example

If we need to record which employee works in which department at which location, all three entities participate in one relationship. The slides map this to a table with a combined primary key (ssn, did, city), plus foreign keys to Employees, Departments, and Locations.

Cardinality trick
Read the relationship in both directions. Ask how many of B can be connected to one A, then how many of A can be connected to one B.

Lecture 13 | Attribute types, weak entities and entity vs attribute

Types of attributes

Type	Meaning	Example
Simple	Cannot be divided into smaller meaningful parts.	Age, salary.
Composite	Can be divided into smaller attributes.	Address = street, city, postcode.
Single-valued	One value for each entity.	One date of birth per person.
Multi-valued	More than one value for one entity.	Several phone numbers.
Derived	Calculated from other attributes.	Age derived from date of birth.
Key	Attribute used to identify an entity uniquely.	StudentID.

Do not mix these pairs
Simple vs composite asks whether it can be broken into parts. Single vs multi-valued asks how many values one entity can have. These are different questions.

Keys in an entity set

A super key is one or more attributes that uniquely determine an entity. A candidate key is a minimal super key. One candidate key is selected as the primary key.

Strong and weak entities

Strong entity: has its own primary key and can be uniquely identified independently.

Weak entity: cannot be uniquely identified using only its own attributes. It needs the primary key of an owner entity, together with its own partial key.

Weak entity example

An Employee may have multiple Dependents. A dependent's name alone may not be unique across all employees. The combination of Employee SSN and Dependent name can identify the dependent.

Weak entity rule	Meaning
Owner to weak entity	One owner can have many weak entities (1:M).
Participation	The weak entity has total participation in its identifying relationship. It must have an owner.
Partial key	The weak entity has a partial key, not a complete independent primary key.
Deletion	When an owner is deleted, its owned weak entities may need to be deleted too. The slides illustrate ON DELETE CASCADE.

SQL pattern from the slides:

```
CREATE TABLE Dep_Policy (
pname CHAR(20),
age INTEGER,
cost REAL,
ssn CHAR(11),
PRIMARY KEY (pname, ssn),
FOREIGN KEY (ssn) REFERENCES Employees(ssn)
ON DELETE CASCADE
);
```

Entity vs attribute: how to decide

The slides give two useful tests:

- An entity can have its own attributes and can participate in relationships. An attribute belongs to one entity.
- If the item has meaningful smaller parts or needs to be related to other things, it may deserve to be an entity.

Example: Product, Supplier, and Address are modeled as entities in the slides because they have their own details. Description, price, phone number, name, street, city, and postcode are attributes. Address is modeled as an entity because it has meaningful parts.

An employee's address can be a simple attribute if only one indivisible value is needed. It may be an entity if an employee can have several addresses or if street, city, and other address parts need separate handling.

Exam decision rule
Make it an entity when it needs its own identity, has its own attributes, participates in relationships, or must occur multiple times. Otherwise it may be an attribute.

Lecture 14 | How to build an ER diagram

ERD symbols to remember

Symbol	Meaning
Rectangle	Entity.
Oval	Attribute.
Underlined attribute	Key attribute.
Diamond	Relationship.
Double oval	Multi-valued attribute.
Dashed oval	Derived attribute.
Double rectangle	Weak entity.
Double diamond	Identifying relationship for a weak entity.
Double line	Total participation.

Worked example: university

Requirement: A university has departments. Each department offers courses. Courses contain modules. Students enroll in a course and take modules. Each module is taught by a lecturer from the appropriate department. Each lecturer tutors a group of students.

Step 1: Identify entities

Department, Course, Module, Student, Lecturer.

Step 2: Identify relationships

Relationship	Plain meaning
Department Offers Course	A department offers courses.
Course Includes Module	A course contains modules.
Student Enrols In Course	A student enrolls in a course.
Student Takes Module	A student takes modules.
Lecturer Teaches Module	A lecturer teaches a module.
Department Employs Lecturer	A department employs lecturers.
Lecturer Tutors Student	A lecturer tutors students.

Step 3: Decide cardinalities from the wording

The phrase 'each department offers several courses' suggests one department to many courses. 'Students enrol in a particular course' suggests each student is assigned to one course in this simplified example, while a course can have many students. 'Students take modules' can be many-to-many if many students take many modules. Always read the exact requirement before assigning a ratio.

Step 4: Draw and verify

Place entity boxes, connect them with named relationships, add attributes, underline keys, and mark cardinality and participation. Then check whether every requirement sentence is represented.

Another example: microscope reservation system

The assignment in the slides describes two user types (administrators and end users), microscopes located in labs, requests assigned to administrators for approval or denial, some microscopes requiring administrator presence, and reservations made from one-hour slots.

Possible entity	Possible attributes / role
User	UserID (PK), name, contact, user type.
Administrator	AdminID (PK), user details. Could be modeled as a user subtype if needed.
Microscope	MicroscopeID (PK), name, requiresAdministrator.
Lab	LabID (PK), location.
ReservationRequest	RequestID (PK), status, submitted date, assigned AdminID, UserID, MicroscopeID.
TimeSlot	SlotID (PK), date, start time, end time. Each slot is one hour.

Likely relationships: one Lab contains many Microscopes; one End User submits many Reservation Requests; one Administrator is assigned many Requests; one Microscope can appear in many reservations over time; one Reservation can occupy one or more TimeSlots. The exact cardinalities and whether Administrator presence is mandatory depend on the business rules.

ERD workflow
Read the problem. Circle nouns to find candidate entities. Find descriptive facts for attributes. Find verbs for relationships. Decide keys. Determine cardinality and participation. Draw the ERD. Check every sentence against the diagram.

Lecture 15 | Entity classification and ERD refinement

Entity or attribute?

Both entities and attributes can appear as nouns in a written description. Use the meaning of the data, not just the grammar, to decide.

- Entities can have attributes and relationships with other entities.
- An attribute belongs to one entity and is usually a descriptive value.

Example from the slides: Product, Supplier, and Address are entities. Product has Description and Price attributes. Supplier has Name and Phone Number. Address has Street Address, City, and Postcode.

Redundant 1:1 relationships

A relationship may be redundant when it is 1:1 and every entity on both sides participates. If every Supplier has exactly one Address and every Address belongs to exactly one Supplier, the two entities may be merged into one entity, carrying all their attributes.

Do not merge entities automatically just because a relationship is 1:1. First check whether every entity participates and whether the two concepts really have the same lifecycle and meaning.

Many-to-many relationships

A many-to-many relationship often needs to be split when translating an ERD into relational tables. Create an associative entity or relationship table containing the keys of both entities, plus any attributes of the relationship.



A student can take many courses; a course can have many students.

For example, Student and Course are M:N. Enrollment resolves this into Student 1:M Enrollment and Course 1:M Enrollment. Enrollment can have attributes such as EnrollmentDate or Grade.

Transactional and component entities

Classification	Meaning	Examples from slides
Transactional entity	Contains measurements that can be summarized.	A sale or purchase transaction with quantity and amount.
Component entity	Describes the who, what, where, when, or why of a business event.	Customer (who), Product (what), Location (where), Period (when).

The slides also mention classifying components that have further classifications. In practice, this means a component may itself be grouped into more detailed categories.

Final ERD design checklist

- Have I identified all entities that need their own identity?

- Have I assigned attributes to the correct entities?
- Does each strong entity have a primary key?
- Have I identified weak entities and their owner keys?
- Are relationship names clear and meaningful?
- Have I checked cardinality in both directions?
- Have I marked optional or total participation correctly?
- Have I checked whether any 1:1 relationship is redundant?
- Have I resolved M:N relationships when translating to relational tables?
- Does the ERD represent every important business rule in the description?

High-yield revision sheet

One-line definitions

Term	Remember it as
Database	Shared, logically related data plus its description.
DBMS	Software that manages and controls access to a database.
Metadata	Data about data.
Schema	Complete logical description of the database.
Subschema / view	A particular user's view or part of the database.
Entity	A thing we store information about.
Attribute	A property of an entity.
Relationship	An association between entities.
Tuple / attribute	Row / column.
Domain	Allowed values for an attribute.
Cardinality / degree of relation	Number of rows / number of columns.
Super key	Any attribute set that uniquely identifies a row.
Candidate key	Minimal super key.
Primary key	Chosen candidate key.
Foreign key	Attribute(s) referencing a candidate key elsewhere.
View	Virtual table derived from base tables.
Index	Structure that speeds up finding rows.
Weak entity	Entity identified using its owner key plus partial key.

Comparisons you should be able to explain

Comparison	Core difference
File system vs DBMS	Separate application-owned files vs centrally managed shared data.
DDL vs DML	Define structure vs read/change data.
External vs conceptual vs internal	User view vs whole logical database vs physical storage.
Logical vs physical independence	Protect user views from logical changes vs protect logical design from storage changes.
Hierarchical vs network	One parent per child vs multiple possible parents.
Object vs relational	Objects/classes vs tables/rows/columns.
Primary vs foreign key	Identifies a row here vs refers to a key in another table.
Simple vs composite attribute	Cannot be divided vs has meaningful subparts.
Single vs multi-valued	One value vs multiple values per entity.
Strong vs weak entity	Own identifier vs needs owner key.
Clustered vs non-clustered index	Physical row order vs separate lookup structure.

Comparison	Core difference
View vs materialized view	Derived when queried vs stored result that must be refreshed/synchronized.

Common exam traps

- Cardinality of a relation means number of rows, not the 1:M relationship ratio.
- Degree of a relation means number of columns. Degree of a relationship means number of participating entity sets.
- A candidate key is minimal. A super key may contain unnecessary extra attributes.
- A primary key cannot be NULL. A foreign key can be NULL if the relationship is optional and the column allows it.
- A foreign key is not automatically unique. Many child rows may reference one parent row.
- An index improves retrieval but can slow inserts and updates.
- A normal view does not store a separate copy of the result. A materialized view does.
- Total participation means every entity must participate. Partial participation means some may not.
- A weak entity needs an owner key for identification and has total participation in its identifying relationship.
- For M:N, use an associative table when mapping to relations.

Self-test | Check your understanding

Try answering each question before checking the answer. These questions are based on the topics in the uploaded lectures.

1. What is the difference between a database and a DBMS?

Answer: A database is the organized data. A DBMS is the software that manages it.

2. Why does data redundancy create inconsistency?

Answer: Because the same fact is stored in multiple places and copies may not all be updated.

3. What is the difference between an entity, attribute, and relationship?

Answer: Entity is the thing; attribute describes it; relationship connects it to another entity.

4. What is the difference between a context diagram and Level 0 DFD?

Answer: Context shows the entire system as one process. Level 0 breaks it into major processes.

5. What is the difference between schema and subschema?

Answer: Schema describes the whole database structure; subschema describes a user/application-specific portion.

6. Which data independence protects conceptual design from storage changes?

Answer: Physical data independence.

7. Which data independence protects user views from conceptual changes?

Answer: Logical data independence.

8. What is the key difference between hierarchical and network models?

Answer: Hierarchical child has one parent; network records can have multiple parents.

9. What do cardinality and degree mean for a relation?

Answer: Cardinality is row count; degree is column count.

10. What makes a candidate key different from a super key?

Answer: A candidate key is a minimal super key with no unnecessary attribute.

11. What is a surrogate key?

Answer: An artificial system-generated identifier used as a primary key.

12. What does referential integrity require?

Answer: A foreign key must match an existing referenced candidate key or be NULL if permitted.

13. Why use a view?

Answer: To customize access, hide data, and simplify queries.

14. What is the trade-off of indexes?

Answer: Faster retrieval, but extra work for inserts and updates.

15. What is a weak entity?

Answer: An entity that needs the owner's primary key plus its own partial key to be identified.

16. When can a relationship be M:N?

Answer: When each side can be associated with multiple occurrences on the other side.

17. What is total participation?

Answer: Every entity in the entity set must participate in the relationship.

18. How do you resolve Student-Course M:N in relational design?

Answer: Create an Enrollment table containing StudentID and CourseID as a combined key, plus any relationship attributes.

Final study method
For every topic, practise saying: definition, why it is used, one example, and one difference from a related concept. If you can do those four things without looking, you understand the topic rather than only memorizing its wording.